

HPE CPP

Week-2 Report

Submitted by: Shubham Kumar

• Type of Streams that QUIC Traffic Carries

- QUIC provides a lightweight, ordered byte-stream abstraction to applications. Streams form the fundamental unit of data transfer within a QUIC connection. Unlike TCP, QUIC supports multiple independent streams within a single connection, allowing concurrent data transfer without head-of-line blocking. This document presents a structured and analytical classification of the types of streams that QUIC traffic carries, based strictly on RFC-defined behaviour.

1. Structural Stream Types in QUIC

Streams in QUIC are identified using a 62-bit Stream ID. The two least significant bits determine the stream type:

- Least Significant Bit (0x01): Identifies initiator (Client = even, Server = odd).
- Second Least Significant Bit (0x02): Identifies direction (Bidirectional = 0, Unidirectional = 1).
- Stream IDs are never reused within a connection.
- Stream IDs increase numerically; out-of-order usage implicitly opens lower IDs.

Bits	Initiator	Direction	Stream Type
0x00	Client	Bidirectional	Client-Initiated Bidirectional
0x01	Server	Bidirectional	Server-Initiated Bidirectional
0x02	Client	Unidirectional	Client-Initiated Unidirectional
0x03	Server	Unidirectional	Server-Initiated Unidirectional

2. Data Transmission Characteristics

QUIC streams encapsulate application data within STREAM frames. Each STREAM frame contains a Stream ID, Offset, Length, and optionally a FIN bit. Streams provide ordered byte delivery within a stream but do not guarantee ordering across different streams.

- Data ordering is guaranteed per stream.
- Endpoints buffer out-of-order data up to flow control limits.
- Duplicate data at the same offset may be discarded.
- Different data at the same offset may trigger `PROTOCOL_VIOLATION`.

Traffic Implication: Because ordering is isolated per stream, multiple streams can operate concurrently and interleave packets on the wire. This enables parallel HTTP transactions, media transfers, and control messaging within the same QUIC connection.

3. Stream Operations

The QUIC specification does not define a strict API. Instead, it defines a set of stream functions that application protocols can rely upon. A QUIC implementation is assumed to provide an interface exposing these operations.

Sending-Side Operations:

- Write data (after reserving stream flow control credit).
- End stream cleanly (STREAM frame with FIN bit set).
- Reset stream abruptly (RESET_STREAM frame if not terminal).

Receiving-Side Operations:

- Read ordered data from the stream.
- Abort reading and request closure (STOP_SENDING frame)

Additionally, applications may request notifications about stream state changes, including:

- Peer opening or resetting a stream
- Peer aborting reading
- Availability of new data
- Flow-control blocking or write availability

Traffic Analysis Relevance: These operations correspond to observable QUIC frames such as STREAM, RESET_STREAM, and STOP_SENDING. Therefore, stream lifecycle events can be inferred from packet captures even though payload content remains encrypted.

- **Identify the streams based on the stream patterns**

Feature / Traffic Characteristic	Video Streaming	Web Browsing	VoIP (Voice/Video Calls)
Typical Packet Size	Mostly large packets close to MTU (≈ 1000 – 1400 bytes)	Mixed packet sizes (small to medium)	Mostly small packets (≈ 60 – 200 bytes)
Packet Size Distribution	Dominated by large packets due to video segments	Irregular distribution depending on webpage objects	Consistent small packets carrying audio frames
Packet Inter-Arrival Time	Bursty; packets arrive in groups when video segments are downloaded	Irregular; depends on user requests and page loading	Very regular intervals (e.g., every 20–30 ms)
Traffic Burst Pattern	Large bursts followed by short idle periods	Short bursts during page load	Minimal bursts; steady packet flow
Packet Rate (Packets/sec)	Medium to high	Low to medium	Low but constant
Flow Duration	Long-lived flows (minutes to hours)	Short flows (seconds)	Long continuous flows during calls
Bandwidth Consumption	High bandwidth usage	Moderate bandwidth usage	Low bandwidth usage
Connection Behaviour	Continuous data delivery for media playback	Multiple short connections for page resources	Continuous low-latency stream
Observed Traffic Pattern	Large packet bursts representing video segment downloads	Small bursts triggered by user browsing activity	Stable stream of small packets
Typical Applications	YouTube, Netflix, Twitch	Web page loading, browsing websites	Zoom, WhatsApp Call, Microsoft Teams

- **Packet size distribution-burst patterns**

- Packet size distribution and burst patterns in QUIC traffic are influenced by how the protocol performs packetization and frame aggregation. A sender can include multiple frames within a single QUIC packet, which allows it to efficiently utilize bandwidth and reduce per-packet overhead. Instead of sending many small packets immediately, the sender may briefly wait to collect several frames and then transmit them together in one packet. This behavior can produce burst-like traffic patterns where multiple frames generated by the application are grouped into fewer, larger packets. However, such waiting must be conservative because delaying packet transmission can increase application-visible latency.
- Another factor affecting packet patterns is stream multiplexing. QUIC enables data from multiple streams to be interleaved within the same packet by placing multiple STREAM frames together. As a result, a single packet may carry data belonging to different streams simultaneously. This improves transmission efficiency and helps utilize available bandwidth effectively. However, if a packet containing frames from several streams is lost, all those streams must wait for retransmission of the lost information before continuing progress, even though QUIC generally avoids head-of-line blocking across independent streams.

- Packet processing and acknowledgment behavior also contribute to observable traffic patterns. Once a packet is received and processed successfully, the receiver acknowledges it using ACK frames that contain the packet number of the received packet. However, acknowledgments are not necessarily sent for every received packet immediately. Instead, receivers may acknowledge multiple packets together to balance timely feedback with reduced processing overhead. This approach helps maintain efficient communication while avoiding excessive acknowledgment traffic that could otherwise consume network resources.
- QUIC also defines constraints on datagram sizes that influence packet size distribution. The protocol assumes that the network path can support a UDP payload of at least 1200 bytes, and endpoints must not use QUIC if the path cannot support this minimum size. Initial packets sent during connection establishment must therefore be padded to at least this size to ensure that the path can handle reasonably large packets. After the connection is established, packet sizes may vary depending on factors such as path capabilities, congestion control behavior, and the amount of application data available for transmission.
- In addition, the protocol allows endpoints to discover larger supported datagram sizes through mechanisms such as path MTU discovery. These mechanisms enable endpoints to probe the network path and adjust the maximum datagram size accordingly, while avoiding IP-layer fragmentation. As a result, the packet size distribution observed in QUIC traffic is not fixed but adapts to network conditions and application behavior. The combination of frame aggregation, multiplexed streams, acknowledgment strategies, and adaptive datagram sizing leads to dynamic traffic bursts and varying packet sizes during the lifetime of a QUIC connection.

• QUIC headers analysis

Header Form (1 bit)	Fixed Bit (1 bit)	Long Pkt Type (2 bits)	Reserved Bits (2 bits)
Packet Number Length (2 bits) — Version (32 bits)			
Destination Connection ID Length (8 bits)			
Destination Connection ID (0..20 bytes)			
Source Connection ID Length (8 bits)			
Source Connection ID (0..20 bytes)			
Additional fields depending on packet type (e.g., Token Length, Token, Length, Packet Number, Payload)			

Header Form (1 bit)	Fixed Bit (1 bit)	Spin Bit (1 bit)	Reserved Bits (2)
Key Phase (1 bit) — Packet Number Length (2 bits)			
Destination Connection ID (0..20 bytes)			
Packet Number (1..4 bytes)			
Payload			

- Unlike traditional protocols such as TCP where much of the header information is visible, QUIC encrypts most transport-layer information including stream identifiers, acknowledgements, and application payload. However, certain header fields remain observable to allow packet routing, connection identification, and protocol interoperability.
- **Types of QUIC Headers**
 - QUIC defines two primary header formats: Long Header and Short Header, each serving a different purpose during the lifecycle of a connection. The long header format is primarily used during the connection establishment phase, where the client and server perform version negotiation, address validation, and cryptographic handshake. Packets that use the long header format include Initial packets, 0-RTT

packets, Handshake packets, Retry packets, and Version Negotiation packets. These packets contain important metadata required to set up the connection and therefore expose more header fields compared to regular data packets. Because of this additional metadata, long header packets are relatively easier for network observers and middleboxes to identify and analyse.

- Once the connection has been successfully established and the cryptographic handshake is completed, QUIC switches to the short header format. Short headers are used during the data transfer phase and primarily carry application data encrypted using the negotiated keys. In this phase, the most common packet type is the 1-RTT packet, which represents normal data exchange between the client and server. Since short headers contain fewer visible fields and most information is encrypted, analysing traffic during this phase becomes more challenging. However, certain observable bits such as the spin bit and connection identifiers still allow limited monitoring and traffic analysis.

- **Important Observable Fields in the QUIC Header**

- **Header Form (1 bit)**
 - The Header Form field is the most significant bit of the QUIC packet header and determines the format of the packet header. If the bit is set to 1, the packet uses the long header format, whereas if the bit is set to 0, the packet uses the short header format. This simple distinction allows network devices to determine whether a packet belongs to the connection establishment phase or the data transmission phase.
 - For this project, the header form bit plays a crucial role in identifying the stage of a QUIC connection. By observing whether packets contain long headers or short headers, the flow detection system can distinguish between handshake-related traffic and regular data transmission traffic. This information is useful for identifying connection initiation patterns and monitoring the transition of a flow from setup to steady-state data transfer.
- **Fixed Bit (1 bit)**
 - The Fixed Bit is always set to 1 in QUIC packets. This field was intentionally designed so that QUIC traffic can coexist with other protocols while still being identifiable by network devices. The presence of a fixed bit ensures that network equipment can recognize QUIC packets even though the majority of the protocol information is encrypted.
 - From a traffic analysis perspective, the fixed bit provides a reliable indicator for detecting QUIC traffic. A simple detection rule can be applied where a UDP packet that contains the fixed bit set to 1 is considered a strong candidate for being a QUIC packet. This helps differentiate QUIC traffic from other forms of UDP-based communication, which typically do not follow the same header structure.
- **Long Packet Type (2 bits)**
 - In long header packets, the Long Packet Type field specifies the type of packet being transmitted during the connection establishment phase. The two-bit value determines whether the packet is an Initial packet, 0-RTT packet, Handshake packet, or Retry packet. Each of these packet types plays a different role in the QUIC connection setup process.

Bits	Packet Type
00	Initial
01	0-RTT
10	Handshake
11	Retry

- This field is particularly useful for understanding the stage of the connection lifecycle. For example, Initial packets indicate the beginning of a new connection attempt, while Retry packets indicate that the server is performing address validation. By analysing the distribution and frequency of these packet types, it becomes possible to detect connection establishment patterns and identify unusual behaviours such as excessive retries or abnormal handshake activity.
- **QUIC Version (32 bits)**
 - The Version field indicates the version of the QUIC protocol being used for the connection. This field allows the client and server to negotiate a mutually supported protocol version during the connection establishment process. If the version field is set to 0, the packet is interpreted as a Version Negotiation packet, which informs the client that the server does not support the initially proposed version.
 - For network analysis, the version field serves as a strong confirmation that the observed traffic belongs to the QUIC protocol. It also provides insight into protocol compatibility and deployment trends, allowing observers to identify which versions of QUIC are being used within the network.
- **Destination Connection ID (DCID)**
 - The Destination Connection ID (DCID) is one of the most important fields in the QUIC header. It uniquely identifies the connection to which the packet belongs. Unlike TCP, which identifies flows using a combination of source IP, destination IP, and port numbers, QUIC uses connection identifiers to maintain connection continuity even when network parameters change.
 - This design allows QUIC connections to survive events such as NAT rebinding or client mobility, where the IP address or port may change during the lifetime of the connection. For the purposes of this project, the DCID plays a critical role in identifying and tracking QUIC flows. By using the DCID as part of the flow key, packets belonging to the same QUIC connection can be grouped together for traffic monitoring and analysis.
- **Source Connection ID (SCID)**
 - The Source Connection ID (SCID) identifies the connection identifier chosen by the sender of the packet. This field enables the receiver to identify which connection identifier should be used when sending packets in the reverse direction.
 - The presence of both DCID and SCID allows bidirectional mapping of the connection and ensures that both endpoints can independently manage their connection identifiers. For flow monitoring systems, the SCID helps improve the accuracy of flow identification by enabling the association of packets transmitted in both directions of a communication session.

- **Spin Bit**
 - The Spin Bit is an optional field present in short header packets, specifically in 1-RTT packets. It was introduced to allow passive measurement of network latency without exposing sensitive information.
 - The spin bit operates by having the server reflect the spin value received from the client, while the client toggles the bit once per round-trip time. Observers on the network can measure the time interval between successive toggles of the spin bit to estimate the Round Trip Time (RTT) of the connection.
 - For this project, the spin bit provides a valuable mechanism for estimating network latency and detecting congestion conditions. By monitoring RTT variations across flows, it becomes possible to support more informed QoS decisions and detect potential performance issues in the network.
- **Encrypted Components of QUIC**
 - Despite the presence of several observable header fields, the majority of QUIC's transport-layer information remains encrypted. This includes CRYPTO frames, stream frames, application data, acknowledgements, HTTP/3 headers, and flow control information. The encryption of these components ensures that user data and protocol operations remain confidential and protected from interception.
 - As a result, traditional network monitoring techniques such as deep packet inspection cannot be applied effectively to QUIC traffic. Instead, traffic analysis must rely on observable metadata from the packet headers and statistical analysis of traffic patterns. This shift represents a fundamental change in how modern encrypted protocols are monitored and managed within networks.
- **Usefulness of QUIC Header Fields for the Project**
 - The observable header fields in QUIC provide sufficient information to support the objectives of this project, which include detecting QUIC flows, distinguishing them from other UDP traffic, and implementing QoS-based rate policing mechanisms.
 - **QUIC Flow Identification**
 - By analyzing fields such as the Fixed Bit, Version field, Header Form, and Connection IDs, it is possible to reliably detect QUIC traffic within UDP flows. These fields provide structural characteristics that uniquely identify QUIC packets, enabling the flow detection system to separate QUIC traffic from other UDP-based protocols.
 - **Distinguishing QUIC from UDP Noise**
 - Random UDP traffic does not follow the structured header format defined by the QUIC protocol. The presence of features such as connection identifiers, long header structures, and the fixed bit provides strong indicators that a packet belongs to QUIC. By combining these features, the detection system can effectively differentiate QUIC flows from unrelated UDP traffic.
 - **Traffic Monitoring and Flow Tracking**
 - Connection identifiers allow packets belonging to the same connection to be grouped together into flows. This enables the monitoring system to track important flow-level metrics such as flow duration, packet transmission rate, and bandwidth consumption. These metrics are essential for understanding traffic behaviour and identifying high-volume or potentially problematic flows.
 - **QoS and Rate Policing**

- The statistical information derived from observable header fields can be used to implement traffic control mechanisms such as Token Bucket policing or Exponentially Weighted Moving Average (EWMA) based rate estimation. These techniques allow the network to regulate bandwidth usage while maintaining fairness among multiple flows. By applying such mechanisms, the system can prevent congestion, manage network resources efficiently, and ensure consistent quality of service for QUIC traffic.

• Encrypted or Protected Details in QUIC Headers

Although QUIC exposes some header fields for routing and connection identification, several fields are protected using QUIC header protection mechanisms. These protections prevent passive observers from easily extracting sensitive protocol information such as packet sequencing or encryption state.

1. Packet Number

The Packet Number field is used in QUIC to maintain packet ordering and detect packet loss during communication. Each packet transmitted in a QUIC connection is assigned a unique packet number that increases monotonically. However, QUIC applies header protection to this field, which masks the actual packet number value. As a result, external observers cannot easily determine the exact sequence of packets exchanged between endpoints.

From the perspective of this project, the protected packet number still indirectly helps in analysing packet flow behaviour and transmission patterns. Although the exact value cannot be observed, variations in packet timing and arrival patterns can still be used to study traffic bursts, retransmission behaviour, and flow characteristics of QUIC connections.

2. Packet Number Length Field

The Packet Number Length field specifies the size of the packet number field within the QUIC header. It determines whether the packet number occupies 1, 2, 3, or 4 bytes. This field is also protected using header protection, meaning that the actual length of the packet number cannot be determined directly by passive observers before the protection is removed by the receiving endpoint.

In the context of this project, the protection of the packet number length prevents precise observation of packet numbering schemes. However, the presence of this field still indicates that the packet participates in QUIC's reliability and loss detection mechanisms, which helps identify legitimate QUIC traffic patterns when distinguishing it from generic UDP traffic.

3. Key Phase Bit

The Key Phase bit is present in the short header of QUIC packets and is used to indicate when the sender switches to a new encryption key during a connection. QUIC periodically updates encryption keys to improve security and reduce the risk of cryptographic attacks. The Key Phase bit allows the receiver to determine which encryption keys should be used to decrypt the packet. This bit is protected using header protection so that observers cannot determine when encryption key updates occur.

For the purposes of this project, although the exact value cannot be observed, the presence of this protected field confirms that the packet belongs to the secure data transmission phase (1-RTT packets). This helps differentiate between handshake packets and regular data packets during QUIC traffic analysis.

4. Reserved Bits

The Reserved Bits are bits within the QUIC header that are reserved for future protocol extensions. According to the QUIC specification, these bits must be set to zero before header protection is applied. After protection, these bits appear masked and cannot be interpreted by observers.

Although these bits do not carry operational information for traffic analysis, their presence is useful in identifying valid QUIC packets. In this project, they help in recognizing the structural format of QUIC headers, which assists in distinguishing QUIC packets from other UDP-based protocols.

5. First Byte Header Control Bits

The first byte of the QUIC header contains several control bits that define packet properties, including fields such as packet number length and other control flags. Portions of these bits are also masked using header protection mechanisms. This ensures that sensitive information related to packet sequencing and encryption states cannot be easily extracted by passive observers.

For the purposes of traffic analysis in this project, the structure of the first byte still helps identify QUIC packet types and header formats, even though some bits remain protected. This information can be used to detect QUIC flows and differentiate them from unrelated UDP traffic.

• Payload Patterns in QUIC Traffic

- QUIC encrypts almost the entire packet payload, including STREAM frames, application data, acknowledgments, and control frames. Because of this encryption, the actual payload content cannot be inspected to directly identify application data or protocol semantics. However, even though the payload itself is encrypted, observable traffic patterns generated by the payload can still provide meaningful information.
- These patterns arise from the behaviour of applications and transport mechanisms, such as how often packets are sent, how large they are, and how traffic bursts occur. For example, applications like video streaming tend to generate large packets in periodic bursts, while real-time communication applications such as VoIP generate small packets at regular intervals. These differences allow analysts to infer the type of traffic without decrypting the payload.
- QUIC also uses packetization and frame aggregation, where multiple frames may be packed into a single packet before transmission. This leads to recognizable patterns in packet sizes and burst behaviour. Even though the encrypted payload hides the actual data, these statistical patterns can be used for traffic classification, anomaly detection, and QoS enforcement.
- Therefore, while the payload cannot be decrypted, payload-generated traffic behaviour still reveals useful metadata such as application type, flow characteristics, and traffic intensity.

• Flow Duration Analysis

- Flow duration refers to the length of time a communication flow remains active between two endpoints. In QUIC traffic analysis, flow duration is an important feature used to classify application types and detect long-lived connections.
- For example, video streaming applications such as YouTube or Netflix typically generate long-lived flows that last several minutes or even hours. These flows continuously transmit data packets as the media stream is buffered and played. In contrast, web browsing traffic usually consists of short-lived flows that terminate once the webpage resources are downloaded.
- To measure flow duration in QUIC, a flow can be identified using metadata such as:

Source IP address

Destination IP address

UDP port numbers

Connection IDs (DCID/SCID)

- By tracking the time difference between the first and last packet of a flow, analysts can determine whether the connection represents a short transaction (e.g., webpage loading) or a persistent stream (e.g., video streaming).
- In the context of this project, identifying long-lived flows is a critical step because these connections typically consume the highest amount of bandwidth over an extended period. By distinguishing these persistent streams from short-lived bursts (like a simple Google search), analysts can effectively implement Traffic Management to prevent network congestion. This identification is essential for enforcing Quality of Service (QoS) policies, which prioritize data like a Zoom call or a VoIP session ensuring that high-bandwidth applications do not degrade the internet speed or stability for other essential services on the same network.

• Upload and Download Traffic Pattern Identification

- Another useful analysis technique involves studying the directional patterns of traffic, particularly upload and download behaviour.
- In many applications, the download traffic volume is significantly larger than the upload traffic volume. For example, in video streaming services, the client primarily downloads large amounts of video data while sending relatively small control or acknowledgment packets. This creates a clear asymmetric traffic pattern, where downstream traffic dominates upstream traffic.
- On the other hand, applications such as video conferencing or VoIP produce a more balanced traffic pattern, where both endpoints continuously send and receive data. This leads to roughly similar packet rates in both directions.
- By comparing the ratio of upstream to downstream packets, analysts can infer the type of application generating the traffic. For instance:

- High downstream traffic with minimal upstream activity often indicates media streaming or content delivery.
- Symmetrical traffic patterns often indicate interactive communication applications.
- In this project, analysing upload/download ratios can help identify traffic categories and enforce appropriate rate-limiting or prioritization policies. By identifying these ratios, the system can automatically enforce Rate-Limiting on non-urgent, high-upload tasks (like cloud backups) to prevent them from choking the network, while simultaneously applying Prioritization to balanced-ratio flows (like VoIP) to ensure low latency. This data-driven approach allows the network to intelligently allocate bandwidth, ensuring that a massive file download doesn't ruin the stability of a critical business call.

• Traffic Simulation Tools for QUIC Analysis

- Traffic simulation tools are used to generate, monitor, and analyse QUIC traffic under controlled network conditions. These tools help simulate realistic application scenarios such as video streaming, web browsing, and real-time communication, allowing analysis of packet size distribution, burst patterns, flow duration, and throughput.
- One commonly used QUIC implementation for traffic generation is quiche, an open-source QUIC and HTTP/3 implementation developed by Cloudflare. Quiche provides client and server implementations that allow developers to generate realistic QUIC traffic and experiment with HTTP/3 communication. It is widely used in testing environments because it closely follows the QUIC RFC specifications and supports modern congestion control mechanisms. By generating QUIC-based traffic flows, quiche helps researchers observe how encrypted traffic behaves under different workloads.
- Another widely used QUIC implementation is ngtcp2, a high-performance QUIC protocol library written in C. It is often used together with HTTP/3 libraries to test QUIC communication at the protocol level. Ngtcp2 allows researchers to analyse connection establishment, handshake behaviour, and congestion control mechanisms defined in the QUIC RFCs. Because it closely follows the official protocol specification, it is commonly used for low-level protocol testing and experimentation.
- For traffic monitoring and packet analysis, tools such as Wireshark are widely used. Wireshark can capture and analyse QUIC packets and display visible header information such as connection IDs, packet lengths, packet types, and handshake messages. Even though the payload is encrypted, Wireshark enables researchers to study packet timing, traffic bursts, and flow characteristics, which are essential for encrypted traffic analysis.
- In addition to Wireshark, packet capture tools like tcpdump are also used to record QUIC traffic into PCAP files for offline analysis. These captured packets can then be examined to extract statistical features such as packet size distribution, inter-arrival time, and flow duration.

• Policy Walkthrough for QUIC Traffic Management

Since QUIC encrypts most packet contents, network policies must rely on observable traffic features rather than payload inspection. A typical policy framework for managing QUIC traffic may involve the following steps:

- **Step 1: QUIC Flow Detection**

The system first identifies QUIC traffic by analysing UDP packets and checking for QUIC-specific header characteristics such as:

- Fixed Bit
- Header Form
- Version field
- Connection ID fields

This step distinguishes QUIC traffic from generic UDP traffic.

- **Step 2: Flow Classification**

Once a QUIC flow is detected, the system analyses traffic features such as:

- packet size distribution
- inter-arrival time
- burst patterns
- flow duration
- upload/download ratio

These features help classify the traffic type, such as streaming, browsing, or real-time communication.

- **Step 3: Policy Enforcement**

Based on the classification results, the network can apply different traffic policies.

Examples include:

- Rate limiting for high-bandwidth streaming flows
- Low latency prioritization for VoIP or real-time communication
- Normal handling for standard browsing traffic

These policies help optimize bandwidth usage and maintain network fairness.

- **Step 4: Performance Monitoring**

Finally, the system monitors performance metrics such as:

- flow throughput
- packet loss
- latency
- congestion indicators

This allows us to evaluate whether the applied policies are effective and adjust them if necessary.